

[H-1] Storing the password on-chain makes it visible to anyone, and no longer private

ROOT source and impact (**Description:** # all data stored on-chain is visible to anyone ,and can be read directly from the blockchain .The PasswordStore::s_password variable is intended to be accessed through the PasswordStore::getPassword function ,which is intended to be only called ,by the owner of the contract)
Impact: Proof of Concept
Proof of Concept: POC
Recommended Mitigation: Due to this ,the overall architecture of the contract should be rethought .One could encrypt the password off -chain and then store the encrypted password on -chain THIS WORLD REQUIRES THE USER TO REMEMBER ANOTHER PASSWORD OFF-CHAIN TO DECRYPT THE PASSWORD HOWEVER YOU'Deploy also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password

Impact & Likelihood : -impact:HIGH -Likelihood:HIGH -Severity:HIGH

[h-2] TITLE 'PasswordStore::setPassword' has no access control ,meaning a non-owner cloud change the password //non-owner owner Description: The setPassword is not control access this setPassword allows only the owner to setPassword a new password

```

1  function setPassword(string memory newPassword) external {
2      // @Audit-There are no Access Controls
3      s_password = newPassword; // 0000000000000000 password
4      emit SetNetPassword();
5  }

```

Impact:   Anyone can set/change the stored password.severely breaking the contract's intended functionality Proof of Concept: Add the following to the PasswordStore.t.sol test file ““ function test_owner_poc_password(address randomAddress) public { //  owner  vm.assume(randomAddress != owner); vm.startPrank(randomAddress); //  owner  string memory attackPassword = “attacker win”;

```

1 //攻击者密码
2 passwordStore.setPassword(attackPassword);
3 // vm.stopPrank(randomAddress);
4 vm.startPrank(owner);
5 //攻击者地址 attacker地址, PasswordStoresol
   攻击者地址 ownergetpassword 的
6 string memory actualPassword = passwordStore.getPassword();
7 //攻击者getPassword 攻击者地址 getPassword 的 stringmemory
   攻击者地址 newPassword
8 assertEquals(actualPassword, attackPassword);

```

```
9 }
```

```
1 Recommended Mitigation:
2 Add an access control conditional to PasswordStore::getPassword
```

```
if (msg.sender!=s_owner){ revert PasswordStore__NotOwner(); } “ Likelihood && Impact : -
Impact:HIGH –Likelihood:HIGH –Severity:HIGH
```

###[L-1] The ‘PasswordStore::getPassword’ natspec indicates a parameter that doesn’t exist,causing the natspec to be incorrect.

Description:

```
1  /*
2    * @notice This allows only the owner to retrieve the password.
3    * @param newPassword The new password to set.
4    */
5    //q ████████████████████getpassword
6    function getPassword() external view returns (string memory) {
```

The ‘PasswordStore::getPassword’ function signature is ‘getPassword()’ while the natspec says it should be ‘getPassword(string)’. //██████████string ██████████ Impact:The natspec is incorrect

Proof of Concept: Recommended Mitigation: Remove the incorrect natspec line ##Likelihood && Impact : –Impact:NONE –Likelihood:NONE –Severity:Information/GAS/Non-

this isn’t a bug but you should know